

Recommender Systems

COMPSCI 753



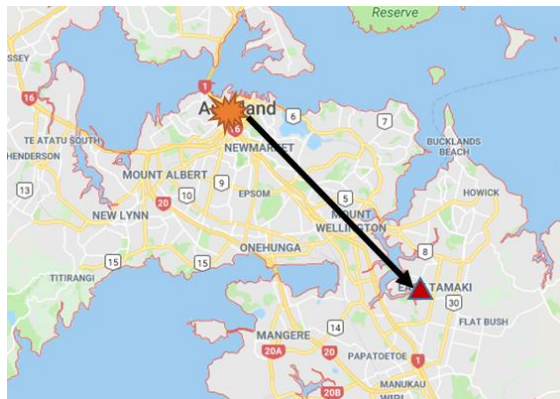
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Outline

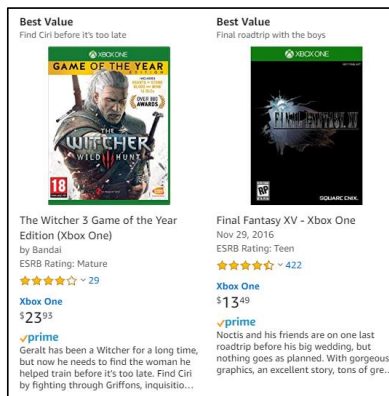
- Context-aware recommendation
- Session-based recommendation

Context-aware recommendation

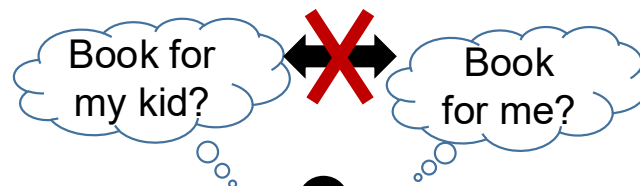
- Traditional model
 - User $\times$ Items $\rightarrow$ Ratings
 - It does not consider the context of the user (e.g., where, when and what?)
- Context is important to make personalized recommendation!



Location



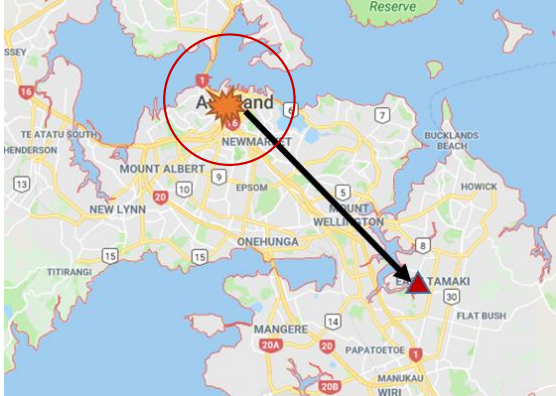
Time



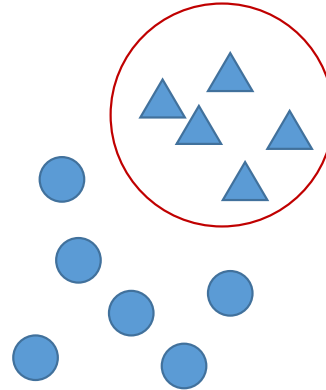
Purpose

Context as filters

- Candidate items that are not within the context can be filtered



Location



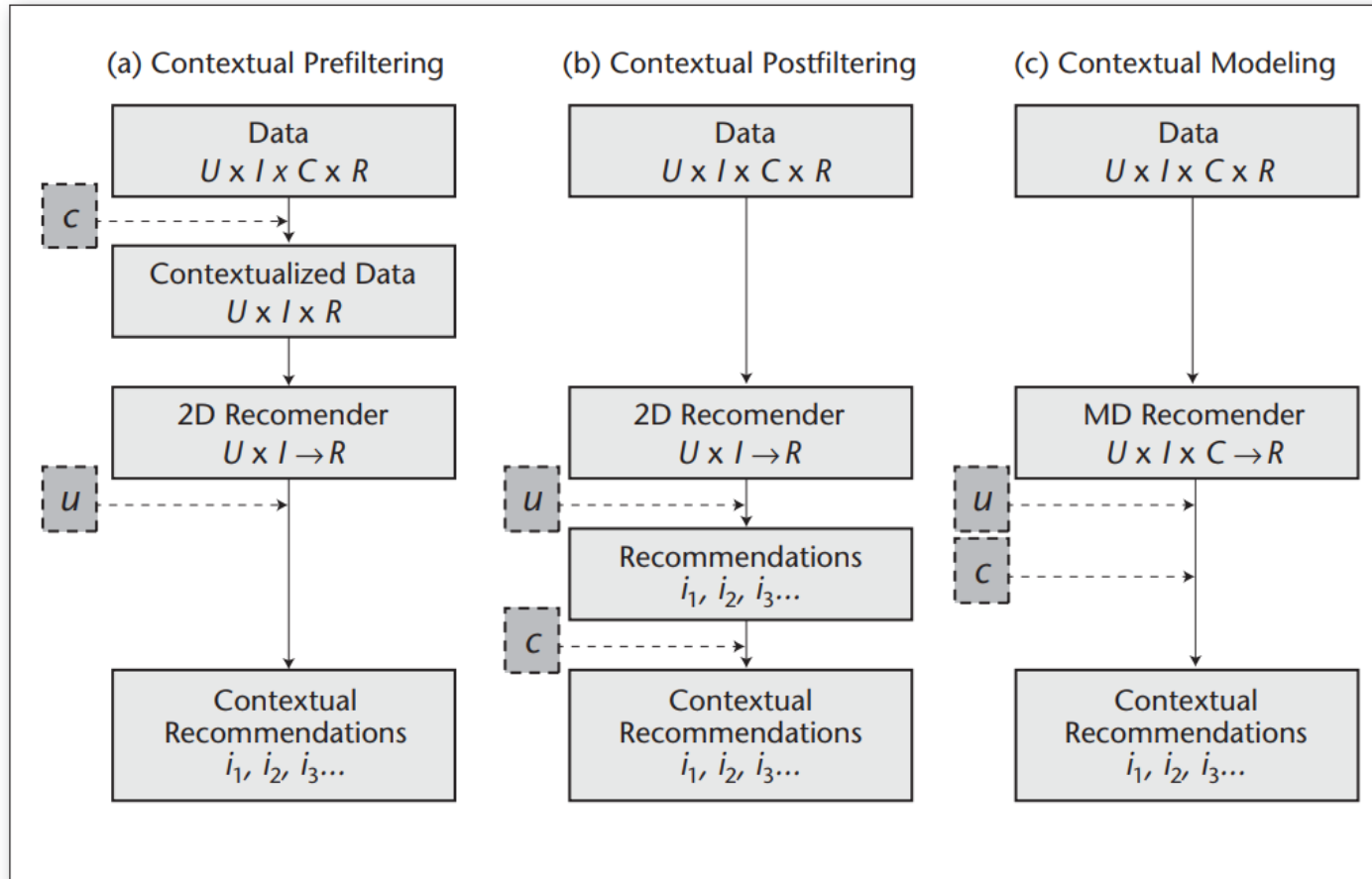
Items with
season/festival
tags

Season/Festival

Context-aware recommendation

- User $\times$ Item $\times$ *Context* $\rightarrow$ Ratings
- Three paradigms for incorporating context information (Adomavicius, 2008)
 - Contextual prefiltering
 - Select relevant data using context
 - Contextual postfiltering
 - Select recommendation results using context
 - Contextual modeling
 - Incorporating context in learning user preferences

Three paradigms for incorporating context information



Adomavicius, et al.
2008

Challenges

- Contextual prefiltering
 - **Over-specification** – the prefiltering makes the data even more sparse!
 - E.g., A user has totally 10 ratings on restaurants, 1 in city central.
- Contextual postfiltering
 - Based on **context similarity**.
 - E.g., filter restaurants that are not in city central from the results
 - It is not efficient for online recommendation
 - It does not uncover the **user's preferences within different contexts**
 - E.g., Tom may prefer French restaurants in weekends, but fast food in weekdays

We will focus on contextual modeling!

Modeling the context with CF

- User-Social-Geo (USG) model (Ye, SIGIR'11) – recommending points of interest (e.g., restaurants)

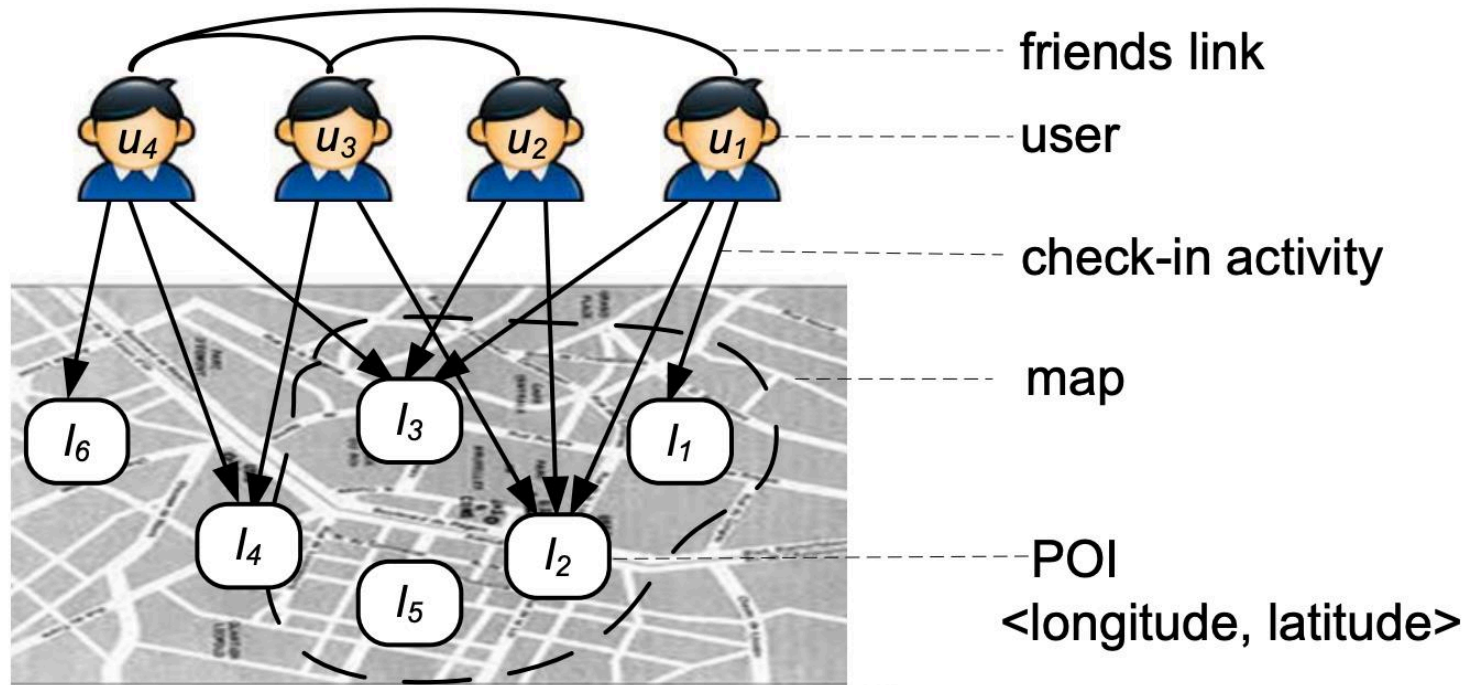


Figure from Ye, SIGIR'11

The USG model

- Three types of information
 - User preference (modeled by collaborative filtering)
 - Social influence from friends (social context)
 - Geographical influence (geo context)

Social influence from friends

$$\hat{c}_{ij}^{(s)} = \frac{\sum_{u_k \in F_i} SI_{ik} c_{kj}}{\sum_{u_k \in F_i} SI_{ik}}$$

Check-in (0,1)

Social influence of user u_k to user u_i

Friends who have closer social tie may have better trust!

$$SI_{ik} = \frac{|F_k| \cap |F_i|}{|F_k| \cup |F_i|}$$

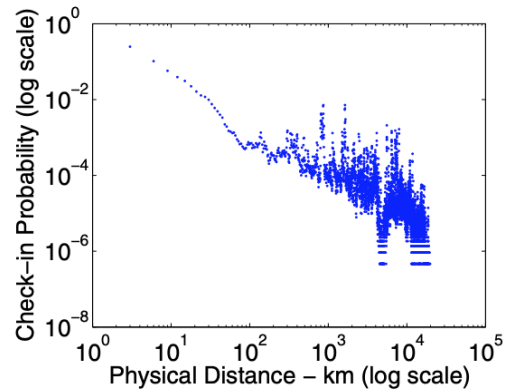
F_k - set of friends of user u_k

The USG model

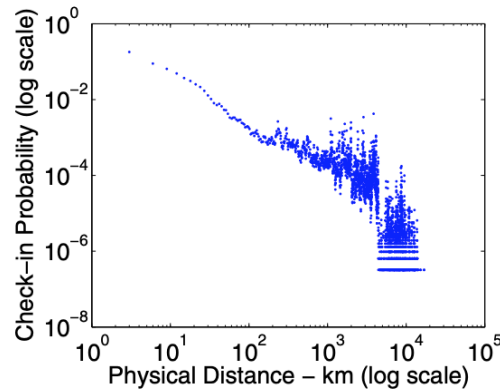
- Three types of information
 - User preference (modeled by collaborative filtering)
 - Social influence from friends (social context)
 - Geographical influence (geo context)

Geographical influence

How likely two check-ins by the same user are within a distance



(a) Foursquare



(b) Whrrl

The USG model

- Three types of information
 - User preference (modeled by collaborative filtering)
 - Social influence from friends (social context)
 - Geographical influence (geo context)

Geographical influence

Let L_i be the set of POIs visited by the user u_i

$$p(L_i) = \prod_{l_n \neq l_m \in L_i} p(l_n, l_m)$$



$$p(l|L_i) = \frac{p(\{l\} \cup L_i)}{p(L_i)} = \frac{\prod_{l_n, l_m \in \{l\} \cup L_i} p(l_n, l_m)}{\prod_{l_n, l_m \in L_i} p(l_n, l_m)} = \prod_{l_n \in L_i} p(l, l_n)$$

Prob that a user visit both locations follows power law distribution.

The probability that the user u_i visit location l , given that the user has visited a set of locations L_i

The USG model

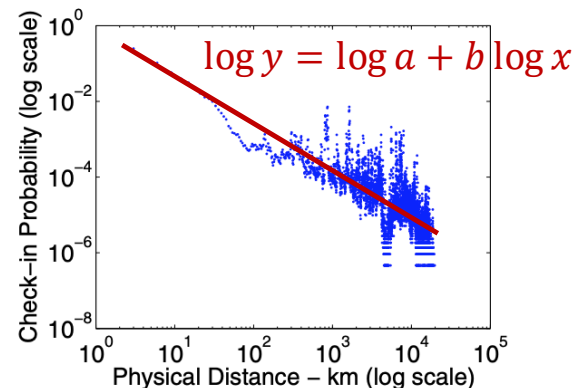
- Three types of information
 - User preference (modeled by collaborative filtering)
 - Social influence from friends (social context)
 - Geographical influence (geo context)

Estimating $p(l_n, l_m)$

Prob that a user visit both locations: $p(l_n, l_m) = y = a \cdot x^b$,
Log-prob $\log a + b \log x$ $x = \text{dist}(l_n, l_m)$

Linear regression problem

$$\mathcal{L} = \frac{1}{2} \sum_{(x,y) \in D_{train}} (\log a + b \log x - \log y)^2$$



The USG model

- Three types of information
 - User preference (modeled by collaborative filtering)
 - Social influence from friends (social context)
 - Geographical influence (geo context)
- Linear fusion the three types of information

$$\hat{c}_{ij} = (1 - \alpha - \beta) \cdot \hat{c}_{ij}^{(CF)} + \alpha \cdot \hat{c}_{ij}^{(s)} + \beta \cdot \hat{c}_{ij}^g \leftarrow p(j|L_i)$$

$\frac{\sum_{u_k \in L_i} w_{ik} c_{kj}}{\sum_{u_k \in L_i} w_{ik}}$ $\frac{\sum_{u_k \in F_i} SI_{ik} c_{kj}}{\sum_{u_k \in F_i} SI_{ik}}$

The USG model

- Pros

- Very flexible to incorporate different context information by adding new estimator of the check-ins
- Can use different approaches to modeling very different contextual information (consider the geo influence model)

- Cons

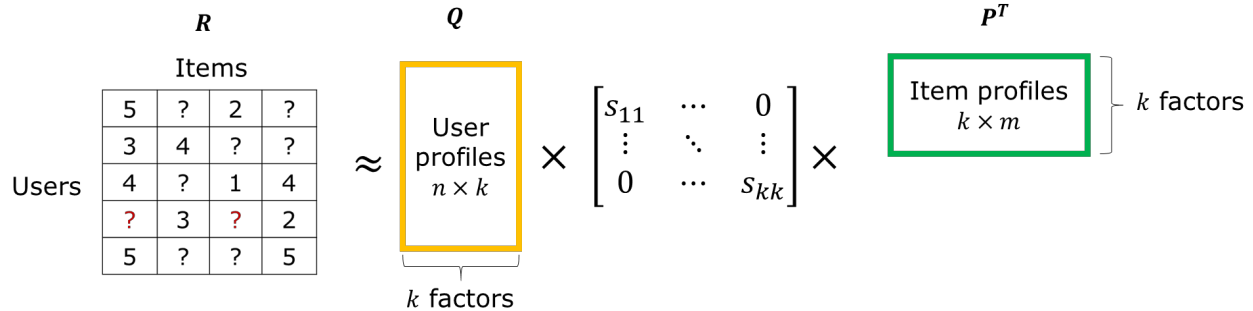
- Cannot capture the interdependency of the context, because each context is modeled independently
- E.g., Tom may prefer French restaurant if he visits city central with his girl friend, but he may prefer fast food during weekdays with his classmates.

Tensor decomposition

■ SVD

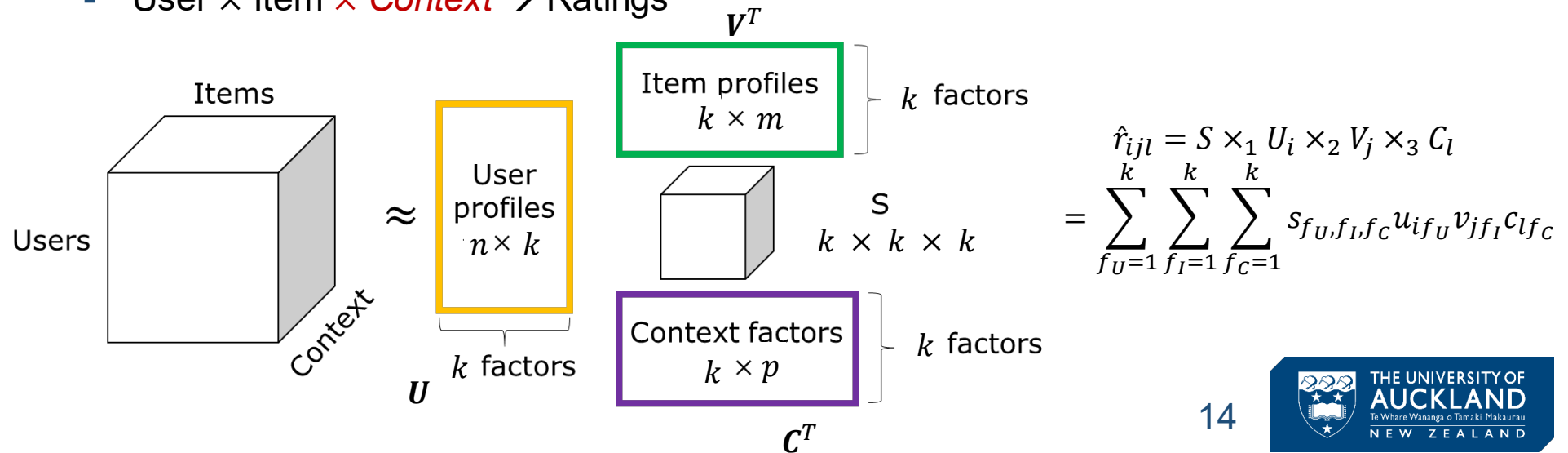
- User $\times$ Item $\rightarrow$ Ratings

$$\hat{r}_{ij} = U_i S V_j^T = \sum_f s_f u_{if} v_{jf}$$



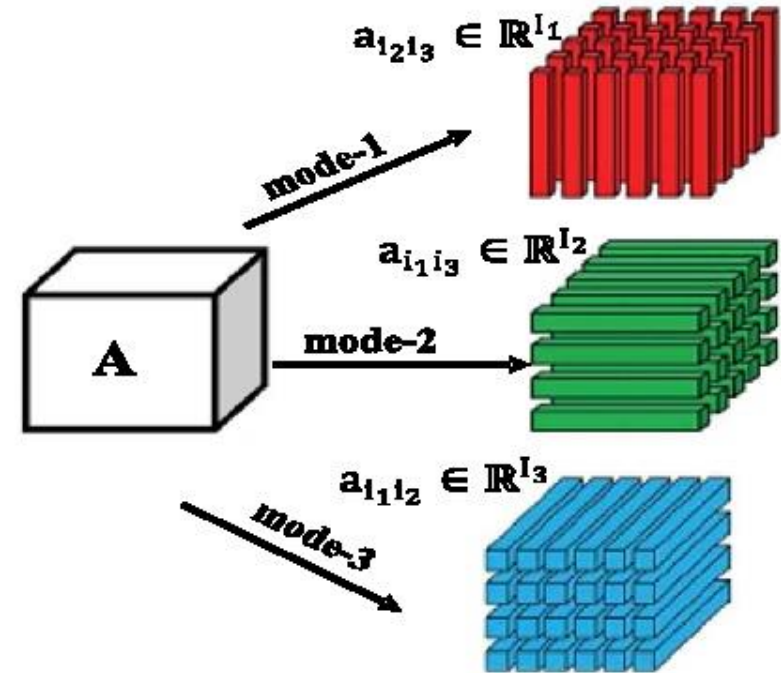
■ Tucker Decomposition (TD)

- User $\times$ Item $\times$ **Context** $\rightarrow$ Ratings



Terminology

- A tensor is N-way array
 - E.g. a matrix is a 2-way array
- A 3-way tensor has three **modes**
 - Mode 1 - Columns
 - Mode 2 - Rows
 - Mode 3 - Tubes



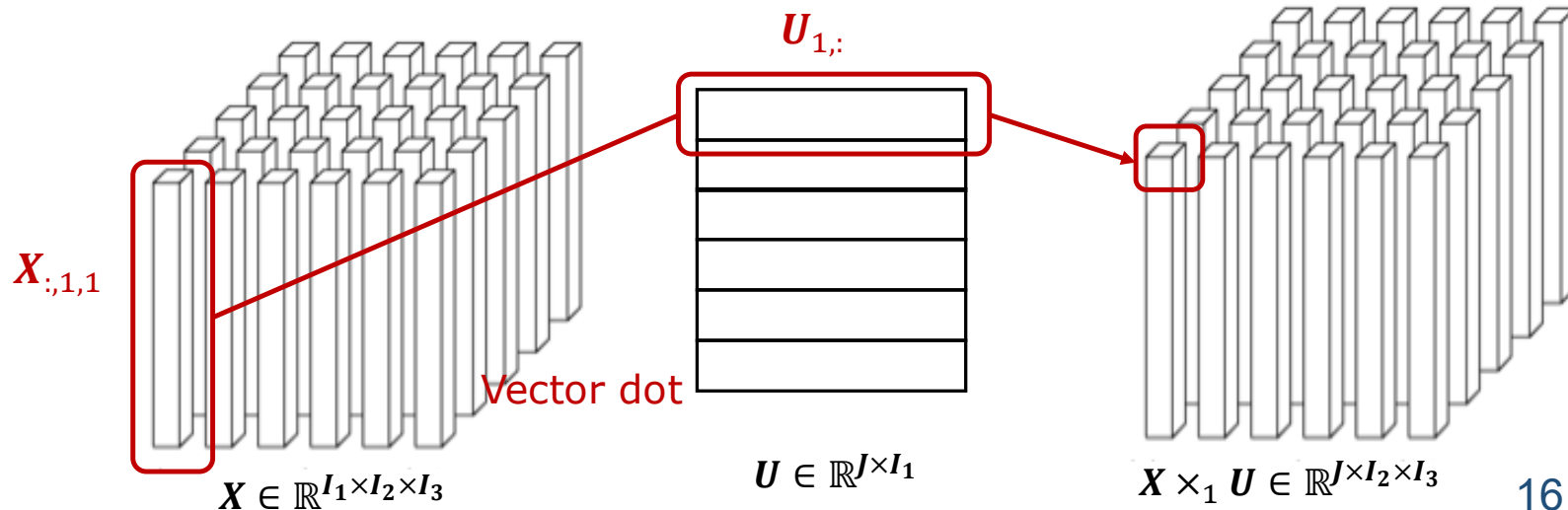
A. Ouamane et al. 2017

n-mode matrix product

- Let X be an N-way tensor of size $I_1 \times I_2 \times \dots I_n \dots \times I_N$, and let U be a matrix of size $J \times I_n$, the n-mode matrix product is:

$$(X \times_n U)_{i_1, i_2, \dots, j, \dots, i_N} = X_{i_1, \dots, i_{n-1}, :, i_{n+1}, \dots, i_N} \cdot U_j = \sum_{i_n=1}^{I_n} x_{i_1, i_2, \dots, i_n, \dots, i_N} \cdot u_{j, i_n}$$

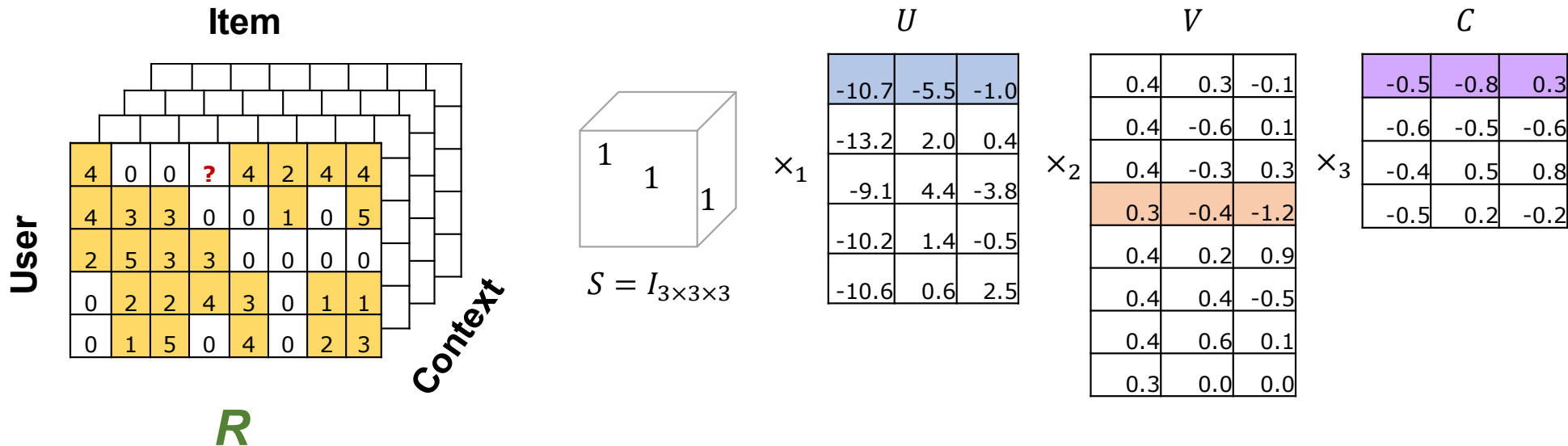
- Example: 1-mode matrix product



Tucker decomposition

- Example

$$\hat{r}_{ijl} = S \times_1 U_i \times_2 V_j \times_3 C_l$$



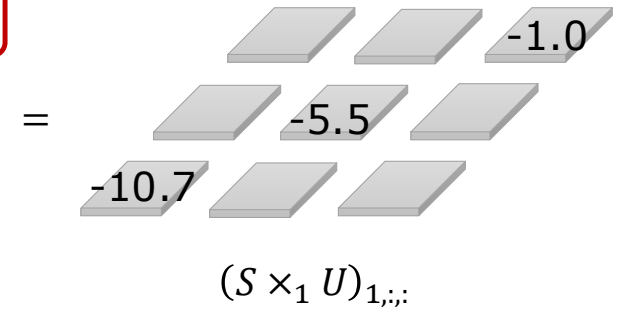
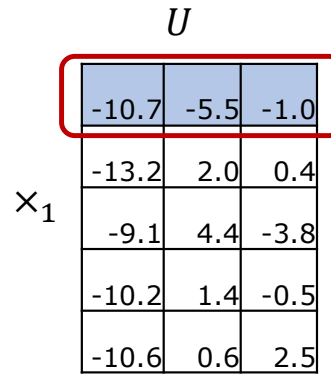
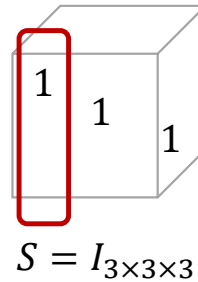
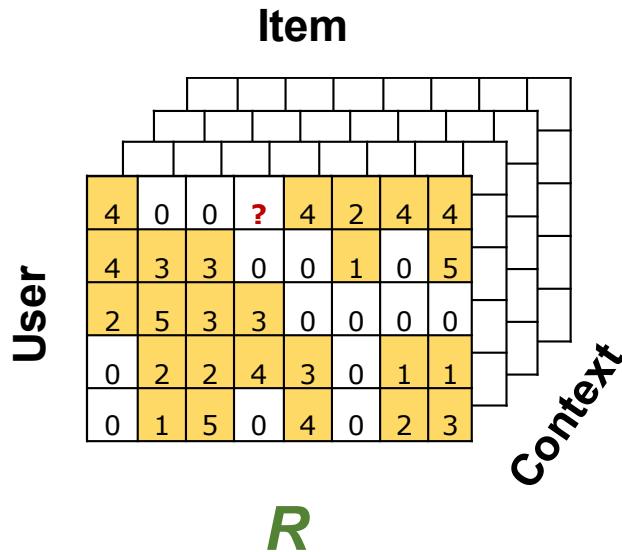
Python package for tensor decomposition

http://tensorly.org/stable/user_guide/tensor_decomposition.html

Tucker decomposition

- Example

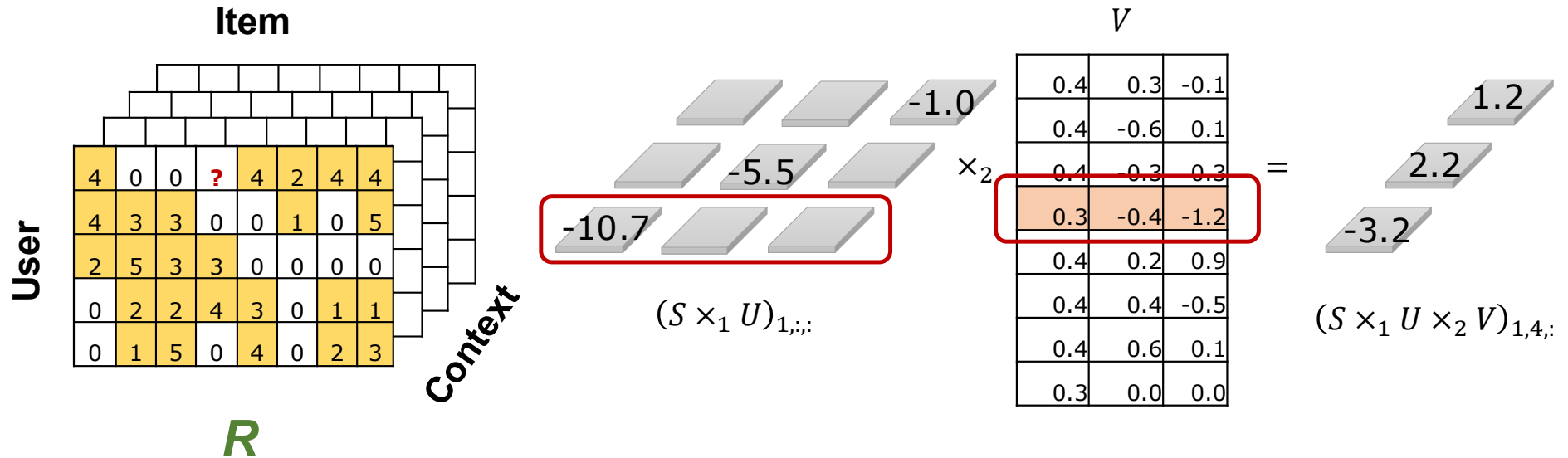
$$\hat{r}_{ijl} = S \times_1 U_i \times_2 V_j \times_3 C_l$$



Tucker decomposition

- Example

$$\hat{r}_{ijl} = S \times_1 U_i \times_2 V_j \times_3 C_l$$



Tucker decomposition

- Example

$$\hat{r}_{ijl} = S \times_1 U_i \times_2 V_j \times_3 C_l$$

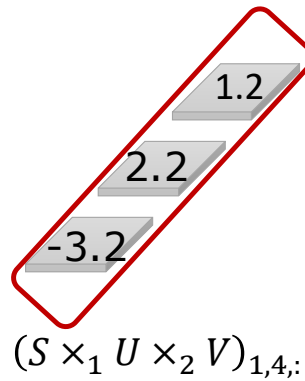
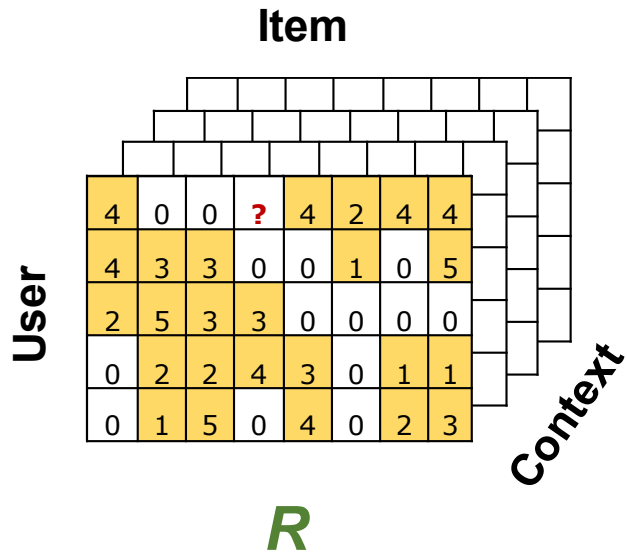


Diagram illustrating the Tucker decomposition components for a recommendation system. The core tensor C is shown as a 3D stack of matrices, with the first matrix (Context 1) highlighted in yellow. The core tensor S is shown as a 3D stack of matrices, with the first matrix (User 1) highlighted in yellow. The core tensor V is shown as a 3D stack of matrices, with the first matrix (Item 1) highlighted in yellow.

The diagram shows the calculation of the predicted rating $\hat{r}_{1,4,1}$ using the Tucker decomposition. The core tensor S is multiplied by the core tensor V (indicated by $\times_3$), resulting in a 3D stack of matrices. The first matrix of this stack is shown as a 3D stack of matrices, with the first matrix (User 1) highlighted in yellow. The core tensor C is then multiplied by this result (indicated by $\times_3$), resulting in a 3D stack of matrices. The first matrix of this stack is shown as a 3D stack of matrices, with the first matrix (Context 1) highlighted in yellow. The final result is a 3D stack of matrices, with the first matrix (User 1) highlighted in yellow.

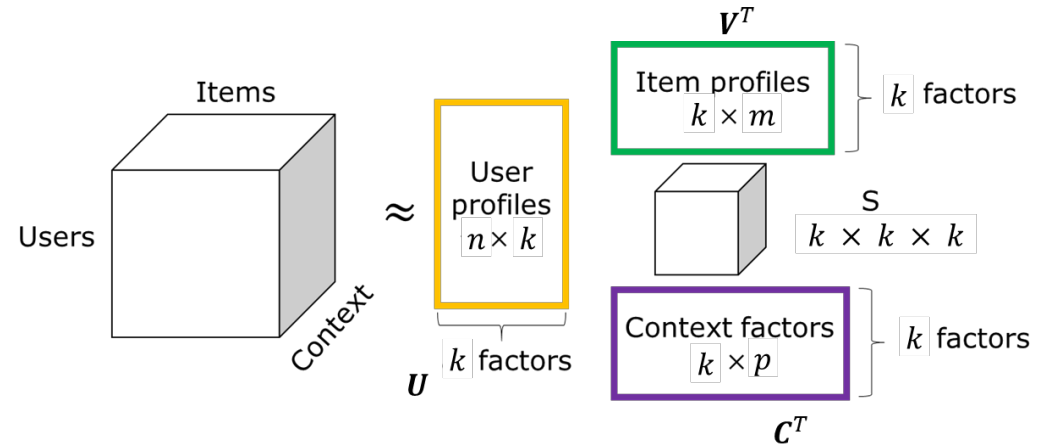
The diagram shows the calculation of the predicted rating $\hat{r}_{1,4,1}$ using the Tucker decomposition. The core tensor S is multiplied by the core tensor V (indicated by $\times_3$), resulting in a 3D stack of matrices. The first matrix of this stack is shown as a 3D stack of matrices, with the first matrix (User 1) highlighted in yellow. The core tensor C is then multiplied by this result (indicated by $\times_3$), resulting in a 3D stack of matrices. The first matrix of this stack is shown as a 3D stack of matrices, with the first matrix (Context 1) highlighted in yellow. The final result is a 3D stack of matrices, with the first matrix (User 1) highlighted in yellow.

Tensor decomposition

■ Tucker Decomposition (TD)

- User $\times$ Item $\times$ **Context** $\rightarrow$ Ratings

$$\begin{aligned}\hat{r}_{ijl} &= S \times_1 U_i \times_2 V_j \times_3 C_l \\ &= \sum_{f_U=1}^k \sum_{f_I=1}^k \sum_{f_C=1}^k s_{f_U, f_I, f_C} u_{if_U} v_{jf_I} c_{lf_C}\end{aligned}$$



■ Canonical Decomposition (CD)

- Let S be a diagonal tensor with all elements equals 1 at the diagonal

$$\hat{r}_{ijl} = \sum_{f=1}^k u_{if} v_{jf} c_{lf}$$

■ Pairwise Interaction Tensor Factorization (PITF)

$$\hat{r}_{ijl} = \sum_{f=1}^k u_{if} v_{jf} + \sum_{f=1}^k u_{if} c_{lf} + \sum_{f=1}^k v_{jf} c_{lf}$$

- Both CD and PITF reduce the complexity of computing one rating from $O(k^3)$ to $O(k)$.
- PITF can explicitly capture pairwise interactions

Factorization machine

- Tensor decomposition
 - Model context, user and item in a single principal way
 - Similar training methods to matrix factorization
 - **Limitations:**
 - The context can neither be real-values nor sets
 - Tensor representation makes the data even sparser
- Factorization machine (Rendle, et al., 2010)
 - Idea:
 - **Consider the rating prediction as a regression problem!**
 - Represent user, item and contexts as features
 - Capture pairwise interactions of features

Factorization machine

- FM treats all inputs (including user, item and context) in the same way
 - Each record from the input can be represented as $[user ; item ; c_1 ; c_2 ; \dots ; c_p]$
 - The records can be represented by real-value, e.g.,
 - Discrete input: User or item $\rightarrow [0 \ 1 \ 0 \ 0]$
 - Context as a set, e.g., the user group $\rightarrow [0 \ 0.5 \ 0.5 \ 0]$
 - Context with real value, e.g., humidity, distance
 - $[user ; item ; c_1 ; c_2 ; \dots ; c_p] \rightarrow$ input feature vector $(x_1, x_2, \dots, x_d)$
- Prediction:

$$\hat{y} = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle v_i, v_j \rangle x_i x_j$$

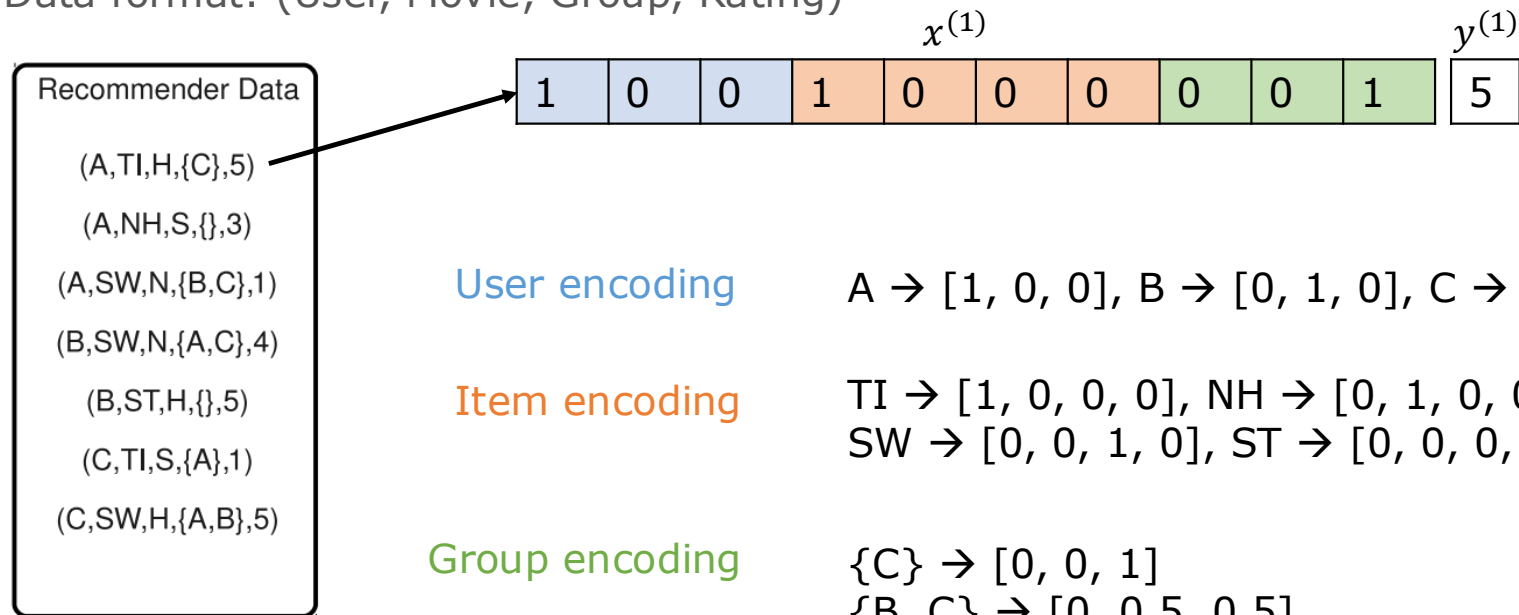
Target (e.g., ratings) Global bias Bias of features

- Factorization of pair-wise interaction
- $\langle v_1, v_2 \rangle$ is often used to denote the **dot product** of v_1 and v_2 in many research papers.

Example

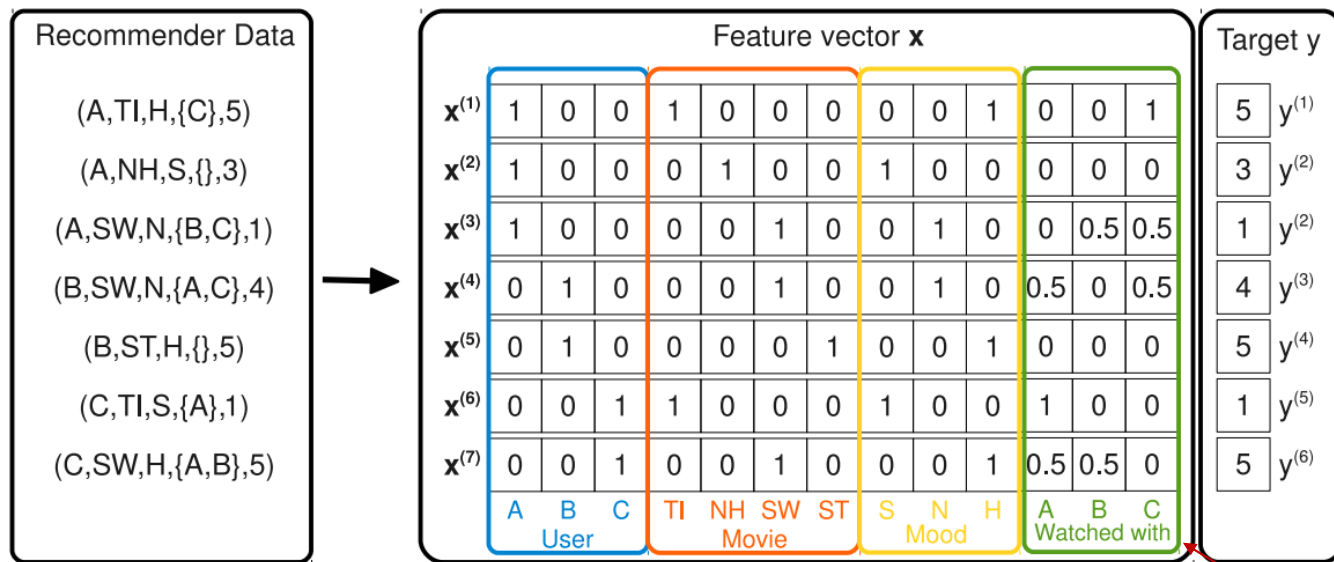
- Movie data – users can view a movie as a group

Data format: (User, Movie, Group, Rating)



Example

- Construct the feature vector



(Figure from Rendle et al., 2010)

- Prediction:

$$\hat{y} = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

More contexts can be easily inserted as columns!

Factorization machine

- Benefit: from modeling higher order tensor to modeling pairs of features
- Computation cost:
 - Original form of FM:

$$\hat{y} = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \quad O(d^2 k)$$

- Equivalent form (Rendle et al. 2010):

$$\hat{y} = w_0 + \sum_{i=1}^d w_i x_i + \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^d v_{if} x_i \right)^2 - \sum_{i=1}^d v_{if}^2 x_i^2 \right) \quad O(dk)$$

Quick Proof (Not examinable):

- $\left(\sum_{i=1}^d v_{if} x_i \right)^2 - \sum_{i=1}^d v_{if}^2 x_i^2 = \sum_{i=1}^d v_{if}^2 x_i^2 + 2 \sum_i^d \sum_{j=i+1}^d v_{if} v_{jf} x_i x_j - \sum_{i=1}^d v_{if}^2 x_i^2 = 2 \sum_i^d \sum_{j=i+1}^d v_{if} v_{jf} x_i x_j$
- $\frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^d v_{if} x_i \right)^2 - \sum_{i=1}^d v_{if}^2 x_i^2 \right) = \sum_{f=1}^k \sum_i^d \sum_{j=i+1}^d v_{if} v_{jf} x_i x_j = \sum_i^d \sum_{j=i+1}^d \left(\sum_f^k v_{if} v_{jf} \right) x_i x_j$
 $= \sum_i^d \sum_{j=i+1}^d \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$

Factorization machine

- Parameters to learn:

- Global bias: w_0
- Weights: $w_{1:d}$
- Latent vector of each feature $\mathbf{v}_{1:d}$

$$\hat{y} = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i+1}^d \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

- Learning the parameters - Stochastic Gradient Descent (SGD)

- Square loss w.r.t. a single record (x, y)

$$\mathcal{L}_{x,y} = \left(y - w_0 - \sum_{i=1}^d w_i x_i - \sum_{i=1}^d \sum_{j=i+1}^d \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \right)^2$$

$$\frac{\partial \mathcal{L}_{x,y}}{\partial v_{if}} = -2(\mathbf{y} - \hat{\mathbf{y}}) x_i \sum_{j \neq i} v_{jf} x_j \quad \frac{\partial \mathcal{L}_{x,y}}{\partial w_i} = -2(\mathbf{y} - \hat{\mathbf{y}}) x_i \quad \frac{\partial \mathcal{L}_{x,y}}{\partial w_0} = -2(\mathbf{y} - \hat{\mathbf{y}})$$

Tensor decomposition v.s. Factorization Machines

- Tensor decomposition:
 - **Data representation:** tensor
 - **Idea:** Decompose the input tensor to latent factors for user, item and context
- Factorization machines:
 - **Data representation:** feature vectors by transactions
 - **Idea:** Regression that captures the pair-wise interactions of features
 - Factorization machines generalize matrix factorization and some specific tensor decompositions (e.g., PITF)

Factorization machines generalize PITF

- Consider the feature vector which concatenates the one-hot encodings of user, item and context

1	0	0	1	0	0	0	0	0	1
u_1	u_2	u_3	i_1	i_2	i_3	i_4	c_1	c_2	c_3

$$y = \underbrace{w_0 + w_{u_1} + w_{i_1} + w_{c_3}}_{\text{Bias term}} + \underbrace{< \mathbf{v}_{u_1}, \mathbf{v}_{i_1} > + < \mathbf{v}_{u_1}, \mathbf{v}_{c_3} > + < \mathbf{v}_{i_1}, \mathbf{v}_{c_3} >}_{\text{Pairwise interactions in PITF}}$$

Summary

- Context-aware recommendation
 - Context modeling with CF - USG
 - Tensor decomposition
 - Factorization machine

Resources

- Datasets:
 - RecSys Challenge 2015: <https://www.kaggle.com/chadgostopp/recsys-challenge-2015>
 - Yelp Dataset Challenge: <https://www.yelp.com/dataset/challenge>
 - MovieLens: <https://www.kaggle.com/prajitdatta/movielens-100k-dataset>
- Readings:
 - Collaborative filtering: <https://lazyprogrammer.me/tutorial-on-collaborative-filtering-and-matrix-factorization-in-python/>
 - Context-aware recommendations <http://www.inf.unibz.it/~ricci/papers/08-Adomavicius.pdf>
 - Factorization machines: <https://www.csie.ntu.edu.tw/~b97053/paper/Rendle2010FM.pdf>
 - Pairwise Interaction Tensor Factorization: <http://www.wsdm-conference.org/2010/proceedings/docs/p81.pdf>
 - Session-based recommendations evaluation <https://arxiv.org/pdf/1803.09587.pdf>